

1 Bevezető

A feladat célja egy olyan neurális hálót tervezni C nyelven, ami pontosságban és sebességben minél jobban közelítse egy tensorflowban vagy pytorchban írt "hivatalos" modellt. Ugyanazt a hálót implementálom C-ben és Tensorflow-ban, majd pontosság és sebesség alapján összehasonlítom őket, és ez alapján eldöntöm, hogy megéri-e egy ilyen kézzel írt, alacsony szintű nyelven írt implementációt használni a profi modell helyett.

Mivel az összehasonlítás inkább magára a programozási nyelvekre éleződött ki, ezért egy egyszerűbb feladatot választottam. A háló Kézzel írt, 28*28 pixeles képeket tanul, amin kézzel rajzolt számjegyek vannak. A modell célja az, hogy minél nagyobb pontossággal eltalálja, hogy a képen milyen szám látható. Másodlagos elvárás, hogy a háló minél gyorsabban tanuljon.

2 Használt adathalmaz

A kézzel írt számjegyekhez az MNIST hivatalos adathalmazát használtam, mely 60000 tanulásra használt, és 20000 tesztelésre szánt mintával rendelkezik:

<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

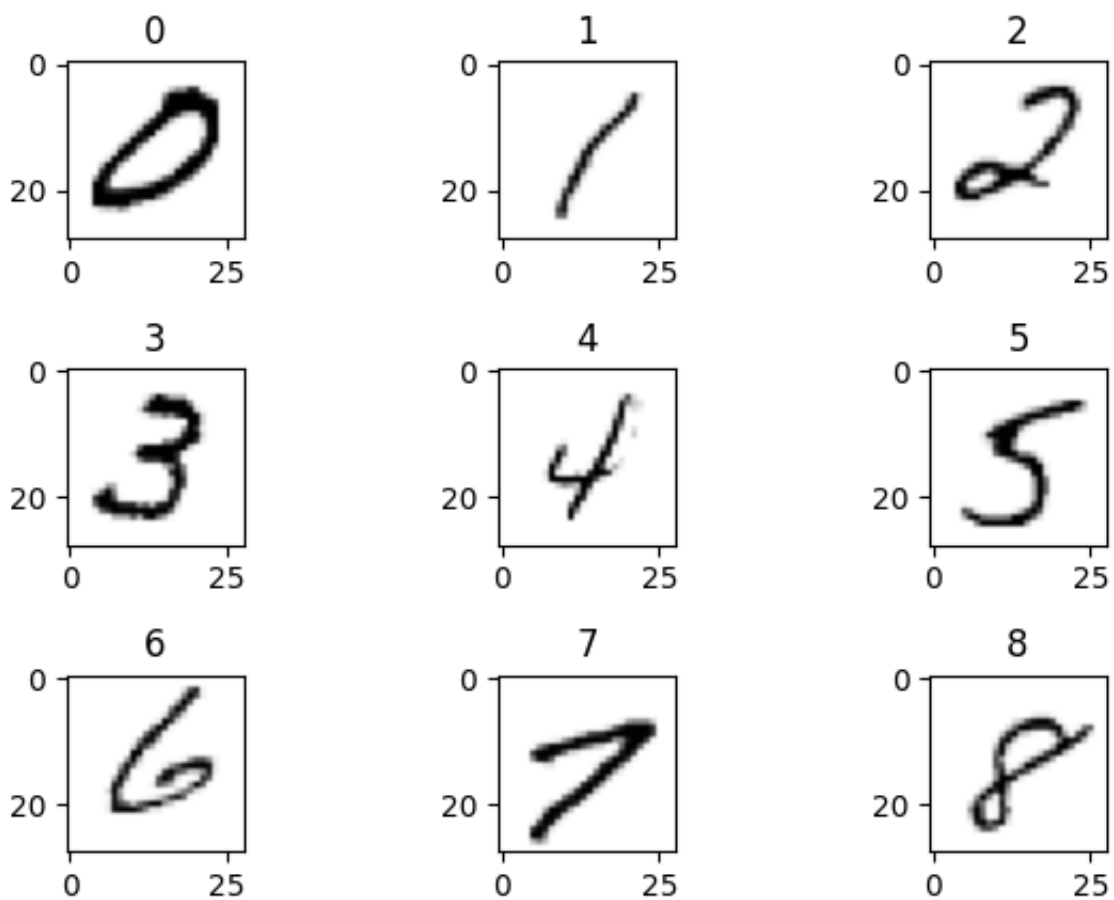


Figure 1: Pár példa az MNIST kézzel írt számjegyeire

3 A háló felépítése

3.1 A háló rétegei

Mindkét esetben a modell 4 rétegből áll.

Az első réteg a bemeneti réteg, ez dolgozza fel a képet. Nem használtam konvolúciót, így a képeket csak kilapítjuk a képeket. Ennek megfelelően az első réteg 784 neuronból áll. Minden neuron egy pixelt reprezentál. Egy neuron erőssége a pixel intenzitásától függ.

A következő két réteg a modell belseje, ezek arra szolgálnak, hogy a modell pontosabban tanuljon. Kísérletezés után a legideálisabb ebben az esetben 64 neuron volt egy rétegben.

Az utolsó réteg a modell kimeneti rétege, ez 10 neuronból áll. Minden neuron sorrendben 0-tól 9-ig a számjegyeket reprezentálja.

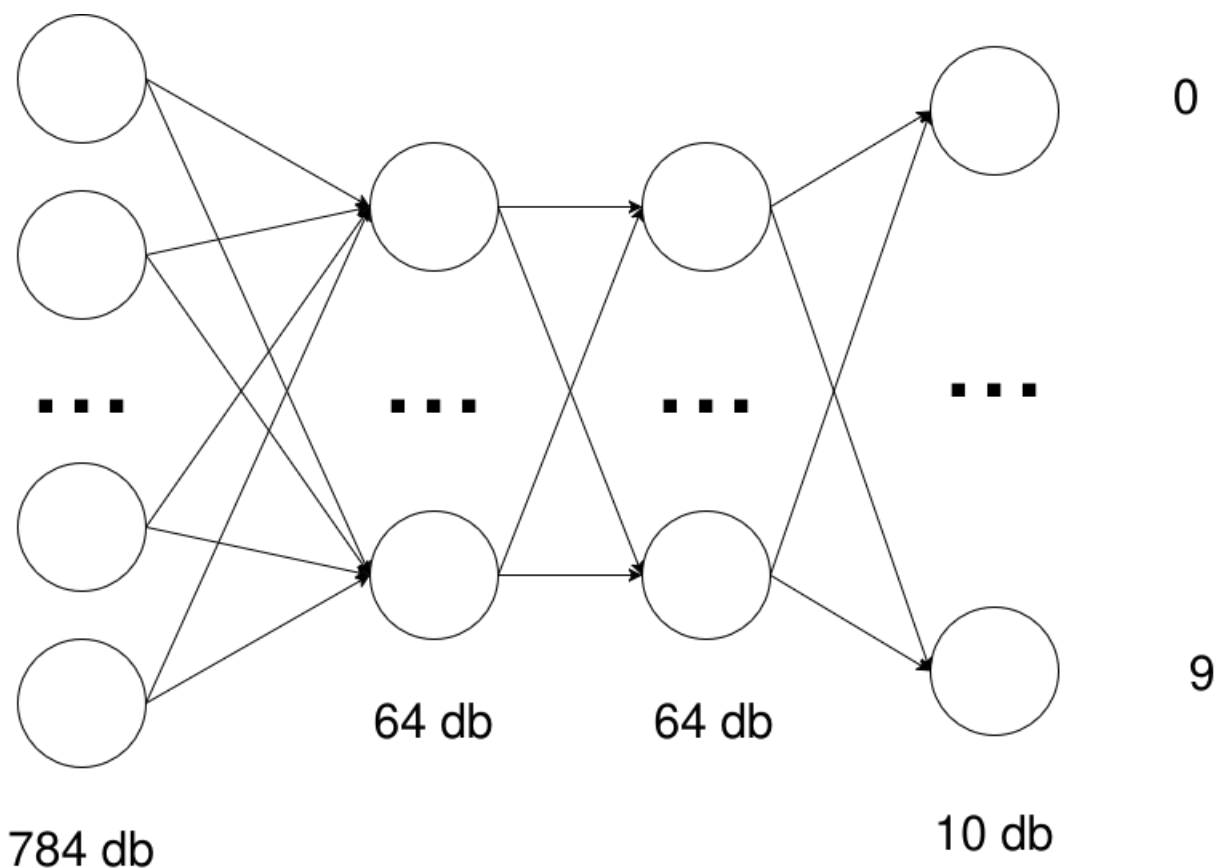


Figure 2: A háló felépítésének vázlata

3.2 Aktivációs függvények

A hálóban a következő aktivációs függvényeket használtam: A bemeneti és a köztes rétegben ReLU-t használtam, hogy tanulásnál ne legyen túl kicsi a derivált.

$$ReLU(x) = \max(0, x)$$

Hogy a kimeneti rétegben lehessen százalékban mérni a pontosságot, az utolsó aktivációs függvényként szigmoidot használok.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

3.3 Optimalizáció

A háló a tanulás finomításához ADAM optimalizációt használ.

Az ADAM első része a momentumot kezeli, célja hogy a lassú gradiens csökkenést gyorsítsa:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

Az ADAM második része az RMSProp, lényege a gradiens sebességének csökkentése túl gyors változás esetén:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

Az optimalizációhoz a β_1 ebben az esetben **0.001**.

3.4 Epochok

A háló több epochon keresztül végzi a tanulást, jelen esetben 5 epochot találtam az optimálisnak. Ha 5-nél kevesebb az epochszám, akkor a pontosság túl alacsony lesz, ha 5-nél több, akkor túl lassú lesz a háló.

4 Implementáció

4.1 C

C-ben a háló rétegei neuronokból álló dinamikus tömbök. A rétegek láncolt lista szerkezetben vannak összekötve.

Minden neuron tárolja a deriváltját a tanulásból, és a következő rétegekbe vezető súlyokat.

A súlyok is tárolják a deriváltjukat.

Az MNIST adathalmazt csv fájlként importáltam, majda programban beolvastam és kilapítottam a modellhez.

4.2 Tensorflow

Ehhez az implementációhoz a tensorflow beépített funkcióit használtam. Az MNIST adathalmazt és az ADAM optimalizációt a Keras könyvtár segítségével töltöttem be és használtam.

5 Eredmények

Mindkét modell futtatása után a következő eredményeket kaptam:

5.1 Gyorsaság

Meglepő módon a C-s implementáció itt elég rosszan szerepelt. A Tensorflow-ban írt modell az 5 epochot 5-ször gyorsabban tanulta meg. A kitekintésben visszatérek erre a tényre, röviden a többszálúság (vagyis C-ben ennek a hiánya) a hiba oka.

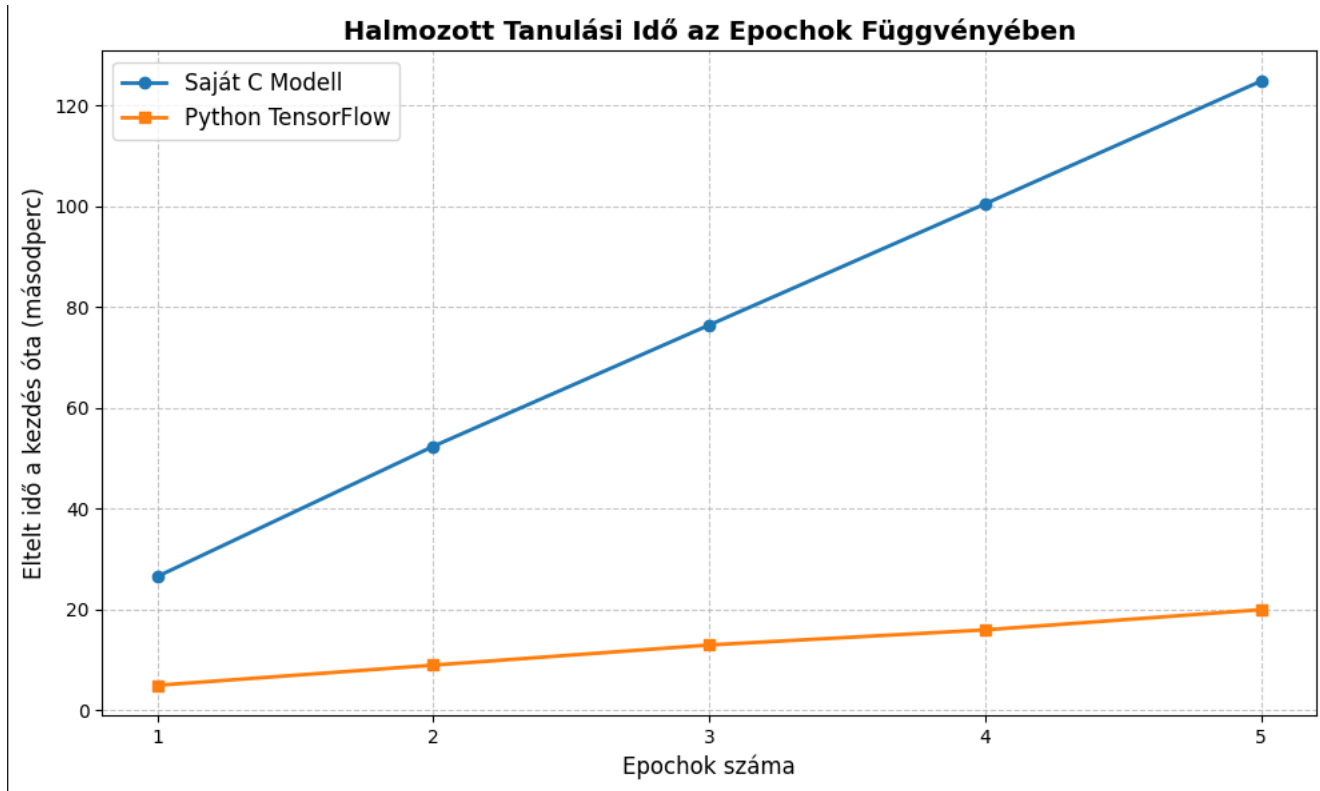


Figure 3: A kettő implementáció gyorsasága epochonként

5.2 Pontosság

Itt a C-s implementáció már jóval jobban szerepelt, minden epoch után csak pár százalékkal volt pontatlanabb, mint a Tensorflowban írt modell.

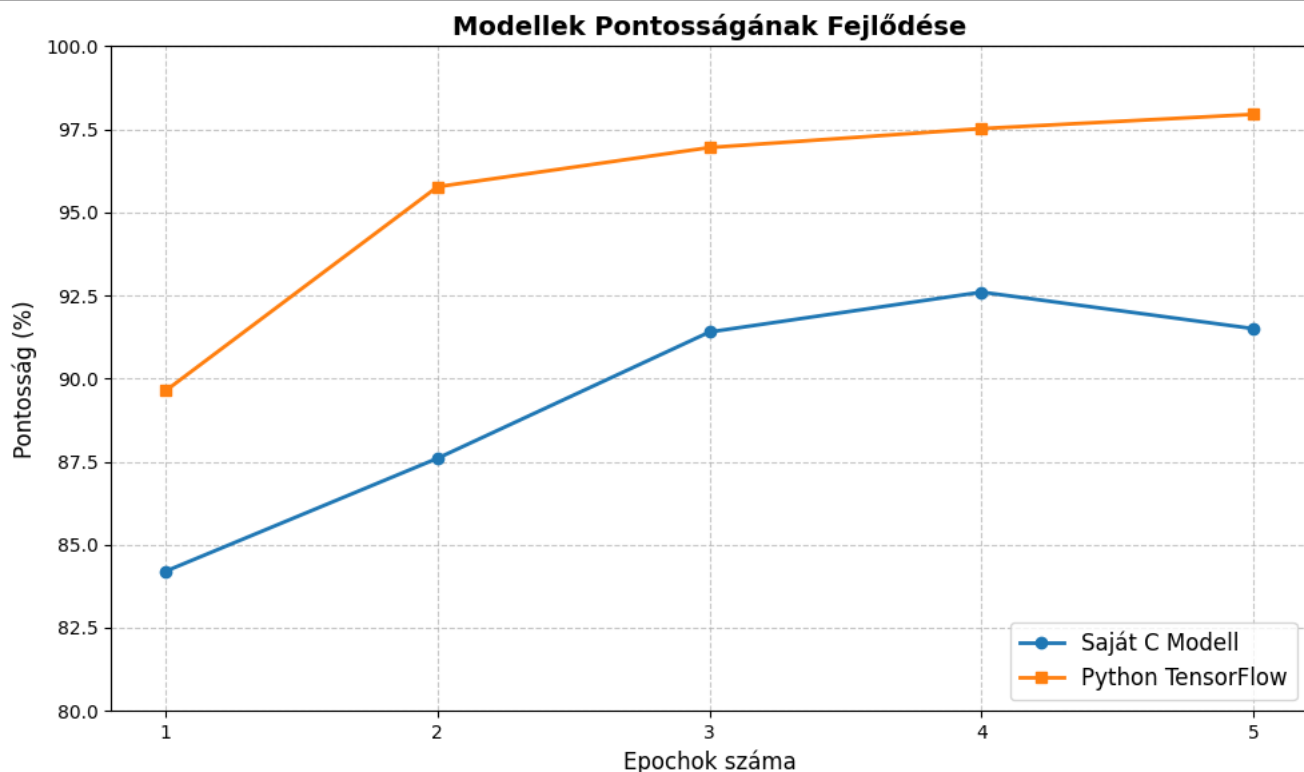


Figure 4: A kettő implementáció pontossága epochonként

6 Kitekintés

A kettő implementáció megírása és a kapott eredmények után a következőkre jutottam:

- **Többszálúság:** Mivel a C egy szálon fut, ezért egyszerre sokkal kevesebb számítást tud végezni, mint a Pythonban írt implementáció, aminek a logikája C++-ban írt, ami támogatja a többszálúságot. Így az első lépés a háló implementálása lenne C++-ban.
- **Mátrixok:** A C-s modellben végül egyesével számoltam az aktivitásokat és a deriváltakat, viszont jóval gyorsabb lett volna, ha számításokat mátrixokkal végzem.
- **Konvolúció:** Mivel nem használtam konvolúciót, ezért a képek lapításánál rengeteg részlet elveszik. Ha a háló konvolúciós lenne, akkor jóval nagyobb lett volna a pontosság.